

What Is Dynamic Programming and What Are Some Common Algorithms?

Dynamic programming is an algorithmic technique that solves complex problems by breaking them down into simpler subproblems and storing the results to avoid redundant calculations. This approach transforms problems that would normally take exponential time into ones that can be solved in polynomial time.

Core Principles of Dynamic Programming

Dynamic programming works when two key conditions are present in a problem.

- **Overlapping Subproblems:** The same smaller problems appear multiple times when solving the larger problem. Instead of recalculating these subproblems repeatedly, we store their solutions.
- **Optimal Substructure:** The optimal solution to the problem contains optimal solutions to its subproblems. This means we can build up the best solution by combining the best solutions to smaller parts.

Let's examine these concepts using the classic "climbing stairs" problem.

The Problem with Naive Recursion

Consider the climbing stairs problem: you're climbing a staircase with `n` steps and can climb either 1 or 2 steps at a time. How many distinct ways can you reach the top?

▼ Example Code

```
def climb_stairs_recursive(n):  
    """Recursive approach"""  
    if n <= 2:  
        return n # Base cases: 1 way for 1 step, 2 ways for 2 steps  
    # To reach step n, we can come from step (n-1) or step (n-2)  
    return climb_stairs_recursive(n-1) + climb_stairs_recursive(n-2)
```

This implementation has exponential time complexity because of massive redundant calculations. When calculating `climb_stairs(5)`, here's what happens:

- `climb_stairs(5)` calls `climb_stairs(4)` and `climb_stairs(3)`
- `climb_stairs(4)` calls `climb_stairs(3)` and `climb_stairs(2)`
- Now `climb_stairs(3)` is calculated **twice**
- `climb_stairs(3)` calls `climb_stairs(2)` and `climb_stairs(1)`
- `climb_stairs(2)` gets calculated **3 times total**

For just `n=5`, we make 9 function calls when we only need 5 unique calculations. As `n` grows, this redundancy explodes exponentially - `climb_stairs(30)` would require a few million function calls. The time complexity becomes $O(2^n)$, making it inefficient and impractical for larger values of `n`.

Dynamic Programming Solutions

Dynamic programming eliminates this redundant computation through two main approaches:

Memoization (Top-Down Approach)

Memoization stores the results of expensive function calls and returns the cached result when the same inputs occur again:

▼ Example Code



```
# Check if we've already calculated this value
if n in memo:
    return memo[n] # Return cached result - O(1) lookup!

# Base cases
if n <= 2:
    return n

# Calculate once and store in memo for future use
memo[n] = climb_stairs_memo(n-1, memo) + climb_stairs_memo(n-2, memo)
return memo[n]
```

Memoization is so much more efficient because each unique value from `1` to `n` is calculated exactly once. When we need `climb_stairs(3)` again, instead of recalculating it (which would trigger more recursive calls), we simply look it up in our memo dictionary in `O(1)` time.

Let's trace through the execution of `climb_stairs(5)` with the top-down approach to see how memoization eliminates redundant work:

▼ Example Code

```
Call: climb_stairs_memo(5)
memo = {} (empty)

Call: climb_stairs_memo(4)
memo = {} (empty)

Call: climb_stairs_memo(3)
memo = {} (empty)

Call: climb_stairs_memo(2) → returns 2 (base case)
Call: climb_stairs_memo(1) → returns 1 (base case)

Result: 2 + 1 = 3
memo = {3: 3} (stored!)

Call: climb_stairs_memo(2) → returns 2 (base case)

Result: 3 + 2 = 5
memo = {3: 3, 4: 5} (stored!)

Call: climb_stairs_memo(3) → returns 3 (FROM MEMO - no recursion!)

Result: 5 + 3 = 8
memo = {3: 3, 4: 5, 5: 8}
```

Efficiency comparison

- **Naive recursive:** Makes 9 function calls with repeated calculations
- **Memoization:** Makes only 5 unique calculations, then reuses stored results
- **Time complexity:** Reduced from `O(2^n)` to `O(n)` since we make only `n` unique calculations
- **Space complexity:** `O(n)` for the memo storage and call stack



Tabulation builds the solution from the ground up, filling a table with solutions to subproblems:

▼ Example Code

```
def climb_stairs_tabulation(n):  
    """Dynamic programming with tabulation"""  
    if n <= 2:  
        return n  
  
    # Create array to store results for all steps from 0 to n  
    dp = [0] * (n + 1)  
    dp[1] = 1 # 1 way to reach step 1  
    dp[2] = 2 # 2 ways to reach step 2  
  
    # Build up the solution iteratively  
    for i in range(3, n + 1):  
        # Ways to reach step i = ways to reach (i-1) + ways to reach (i-2)  
        dp[i] = dp[i-1] + dp[i-2]  
  
    return dp[n]
```

Tabulation eliminates recursion entirely by building the solution iteratively from the smallest subproblems up to the target.

Let's see the bottom-up approach in action to see how we build the solution systematically. Here's the iterative construction for

`climb_stairs(5)`:

▼ Example Code

```
Initial state:  
dp = [0, 1, 2, 0, 0, 0]  
[0, 1, 2, 3, 4, 5] ← indices (step numbers)  
  
Step by step construction:  
  
i = 3:  
dp[3] = dp[2] + dp[1] = 2 + 1 = 3  
dp = [0, 1, 2, 3, 0, 0]  
  
i = 4:  
dp[4] = dp[3] + dp[2] = 3 + 2 = 5  
dp = [0, 1, 2, 3, 5, 0]  
  
i = 5:  
dp[5] = dp[4] + dp[3] = 5 + 3 = 8  
dp = [0, 1, 2, 3, 5, 8]  
  
Final result: dp[5] = 8
```

Key advantages of tabulation

- **No recursion overhead:** Unlike memoization, there's no recursive call stack.
- **Predictable execution:** We calculate values in a predetermined order (1, 2, 3, 4, 5...).



```
def climb_stairs_optimized(n):
    if n <= 2:
        return n

    prev2, prev1 = 1, 2 # Only store last two values
    for i in range(3, n + 1):
        current = prev1 + prev2
        prev2, prev1 = prev1, current
    return prev1
```

Efficiency comparison

- **Naive recursive:** 9 function calls for $n=5$, exponential growth.
- **Tabulation:** 3 simple additions for $n=5$, linear growth.
- **Time complexity:** $O(n)$ instead of $O(2^n)$.
- **Space complexity:** $O(n)$ for the array, or $O(1)$ with optimization.
- **Predictable performance:** No risk of stack overflow for large inputs.

Both approaches reduce the time complexity from exponential $O(2^n)$ to linear $O(n)$, a dramatic improvement that makes the difference between solving the problem in milliseconds versus waiting years for larger inputs.

Real-World Applications

Dynamic programming has widespread applications in computer science and beyond:

- **Route Optimization:** GPS systems use dynamic programming algorithms to find shortest paths between locations.
- **Text Processing:** Spell checkers and autocomplete features often rely on dynamic programming to calculate edit distances between words.
- **Financial Modeling:** Investment strategies and portfolio optimization frequently employ dynamic programming techniques.
- **Resource Allocation:** The knapsack problem and its variants appear in scheduling, budgeting, and resource management.

Practical Example: Coin Change Problem

The coin change problem is a classic coding challenge that, when solved using dynamic programming, demonstrates both of DP's key principles: optimal substructure and overlapping subproblems.

The coin change problem asks, "What's the minimum number of coins needed to make a target amount?"

Here's one solution using dynamic programming:

▼ Example Code

```
def min_coins(amount, coins):
    """Find minimum number of coins needed to make the given amount"""
    # Initialize dp array with "infinity" - represents impossible to make
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0 # Base case: 0 coins needed for amount 0

    # For each amount from 1 to target amount
    for i in range(1, amount + 1):
        # Try each coin denomination
        for coin in coins:
            if coin <= i: # Can only use coin if it doesn't exceed current amount
```



```
# Return result if possible, -1 if impossible
return dp[amount] if dp[amount] != float('inf') else -1

# Example usage:
# coins = [1, 3, 4], amount = 6
# dp[6] = min(dp[5]+1, dp[3]+1, dp[2]+1) = min(3+1, 1+1, 2+1) = 2
# Result: 2 coins (3 + 3)
```

And here's how the dynamic programming coin change algorithm works step by step for `coins = [1, 3, 4]`, `amount = 6`:

▼ Example Code

```
Initial state:
dp = [0, ∞, ∞, ∞, ∞, ∞, ∞]
    [0, 1, 2, 3, 4, 5, 6] ← amounts

Building up the solution:

For amount = 1:
  Try coin 1: dp[1] = min(∞, dp[0] + 1) = min(∞, 0 + 1) = 1
  dp = [0, 1, ∞, ∞, ∞, ∞, ∞]

For amount = 2:
  Try coin 1: dp[2] = min(∞, dp[1] + 1) = min(∞, 1 + 1) = 2
  dp = [0, 1, 2, ∞, ∞, ∞, ∞]

For amount = 3:
  Try coin 1: dp[3] = min(∞, dp[2] + 1) = min(∞, 2 + 1) = 3
  Try coin 3: dp[3] = min(3, dp[0] + 1) = min(3, 0 + 1) = 1
  dp = [0, 1, 2, 1, ∞, ∞, ∞]

For amount = 4:
  Try coin 1: dp[4] = min(∞, dp[3] + 1) = min(∞, 1 + 1) = 2
  Try coin 3: dp[4] = min(2, dp[1] + 1) = min(2, 1 + 1) = 2
  Try coin 4: dp[4] = min(2, dp[0] + 1) = min(2, 0 + 1) = 1
  dp = [0, 1, 2, 1, 1, ∞, ∞]

For amount = 5:
  Try coin 1: dp[5] = min(∞, dp[4] + 1) = min(∞, 1 + 1) = 2
  Try coin 3: dp[5] = min(2, dp[2] + 1) = min(2, 2 + 1) = 2
  Try coin 4: dp[5] = min(2, dp[1] + 1) = min(2, 1 + 1) = 2
  dp = [0, 1, 2, 1, 1, 2, ∞]

For amount = 6:
  Try coin 1: dp[6] = min(∞, dp[5] + 1) = min(∞, 2 + 1) = 3
  Try coin 3: dp[6] = min(3, dp[3] + 1) = min(3, 1 + 1) = 2
  Try coin 4: dp[6] = min(2, dp[2] + 1) = min(2, 2 + 1) = 2
  dp = [0, 1, 2, 1, 1, 2, 2]

Final result: dp[6] = 2 (achieved with coins 3 + 3)
```



we know the minimum coins for amount 3 is 1, then one way to make amount 6 is to use that solution plus one more coin of value 3.

Without DP, we'd need to try every possible combination of coins - an exponential number of possibilities. With DP, we build up the solution systematically:

- **Time complexity:** $O(\text{amount} \times \text{number of coins})$ instead of exponential.
- **Space complexity:** $O(\text{amount})$ for the `dp` array.
- **No redundant work:** Each subproblem (finding minimum coins for each amount) is solved exactly once.
- **Reusable results:** Once we know the minimum coins for amount 3, we use this knowledge for all larger amounts that can benefit from it.

When to Use Dynamic Programming

Dynamic programming is effective when:

- The problem can be broken down into overlapping subproblems.
- The problem exhibits optimal substructure.
- A naive recursive solution would involve repeated calculations.
- You need to optimize for time complexity at the cost of space complexity.

Common dynamic programming patterns include optimization problems (finding minimum/maximum values), counting problems (number of ways to achieve something), and decision problems that can be broken down into smaller decisions.

Dynamic programming transforms complex problems into manageable ones by systematically storing and reusing solutions to subproblems. Understanding this technique opens the door to solving a wide range of computational challenges efficiently.

Questions

What are the two key conditions that must be present for dynamic programming to be applicable?

- ☐ Fast execution and low memory usage.
- ☐ Overlapping subproblems and optimal substructure.
- ☐ Recursive calls and iterative loops.
- ☐ Memoization and tabulation.

What is the main difference between memoization and tabulation approaches in dynamic programming?

- ☐ Memoization uses more memory than tabulation.
- ☐ Tabulation is always faster than memoization.
- ☐ Memoization is top-down while tabulation is bottom-up.
- ☐ Memoization only works with recursive functions.



- ☐ $O(n)$ because it calculates each step once.
- ☐ $O(n^2)$ because it uses nested loops.
- ☐ $O(2^n)$ because it recalculates the same subproblems multiple times.
- ☐ $O(\log n)$ because it divides the problem in half.

[Check your answer](#)[Ask for Help](#)